PES UNIVERSITY, RR CAMPUS

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING (AIML)

LAB 7: FUZZING FOR INPUT VALIDATION BUGS

Dynamic Property-Based Testing, Test-Case Shrinking & Defensive Code Hardening

Student: Dharshini V | SRN: PES1UG24AM428 | Course: Software Engineering Lab

Assigned Repo: SETAPESU26/L6_L7_56_course_registration | Personal Repo: DharshiniPES/L6_L7_56_course_registration

1. Executive Summary & Fuzz Testing Principles

Fuzz testing (or fuzzing) is a dynamic testing technique that executes software with randomized, unexpected, and malformed data inputs to discover latent defects, edge-case crashes, memory corruptions, and security vulnerabilities. Unlike traditional manual unit testing—which verifies 'happy paths' and anticipated boundary values—fuzzing tests invariants and properties against inputs human developers rarely imagine.

In this lab, property-based fuzz testing was conducted using the Hypothesis library for Python on the assigned Course Registration Parser module (course_registration_parser.py / processor.py). Hypothesis automatically generated thousands of randomized Unicode string and null combinations using the composite strategy `st.text() | st.none()`. When defects were detected, Hypothesis systematically shrank failing inputs to their minimal reproducing edge case, allowing precise root-cause analysis and the implementation of defensive programming guards.

Testing Dimension	Traditional Example-Based Unit Testing	Property-Based Fuzz Testing (Hypothesis)
Test Generation	Developer manually authors fixed, discrete inputs.	Hypothesis algorithmically synthesizes hundreds of randomized inputs.
Scope of Inputs	Typically happy-path and obvious boundary values.	Malformed data, nulls, empty strings, Unicode, control characters, extremes.
Failure Minimization	Raw crashing input returned; often large and noisy.	Automated 'Shrinking' isolates the minimal falsifying example (e.g. '', None).
Defect Detection	Confirms expected behaviors against known scenarios.	Uncovers unexpected exceptions, unhandled NoneTypes, and parsing crashes.
Regression Replay	Tests pass/fail identically on static values.	Failing examples are cached locally (.hypothesis/) for deterministic replay.

2. System Under Test: course_registration_parser.py

The Course Registration Parser module processes user-supplied text during academic course enrollment. The application exposes three primary parsing routines:

Function Identifier	Expected Input Format	Expected Normal Output	Original Naive Assumption
format_course_code(code)	'cs 301' (space separated)	'CS301' (uppercase normalized)	Assumes exactly two space-separated words (dept, number).
total_credits(text)	'4,3,4' (comma separated)	11 (integer credit sum)	Assumes non-empty string of pure valid whole numbers.
parse_student_record(line)	'SRN|name|status'	{'srn': ..., 'name': ..., 'status': ...}	Assumes all 3 pipe-separated fields are

always present.

3. Fuzz Testing Findings & Defect Analysis (Buggy Module)

Executing the Hypothesis test suite (@given(st.text() | st.none())) against the initial unhardened module immediately caused all three functions to crash. Hypothesis automatically reduced the complex generated strings down to minimal falsifying examples. The table below documents the discovered vulnerabilities:

Function Under Test	Shrunk Falsifying Input	Exception Raised	Root Cause & Architectural Flaw
format_course_code(code)	code = None	AttributeError: 'NoneType' has no attribute 'split'	Directly invoked .split() on input without None or type guard.
format_course_code(code)	code = '' (empty string)	ValueError: not enough values to unpack (expected 2, got 0)	Unconditionally unpacked dept, number = code.split() without checking length.
format_course_code(code)	code = 'CS'	ValueError: not enough values to unpack (expected 2, got 1)	Crashed when input contained only department without course number.
format_course_code(code)	code = 'cs 301 lab extra'	ValueError: too many values to unpack (expected 2)	Crashed when input contained more than two whitespace-delimited tokens.
total_credits(text)	text = None	AttributeError: 'NoneType' has no attribute 'split'	Invoked .split(',') on None without parameter validation.
total_credits(text)	text = '' (empty string)	ValueError: invalid literal for int() with base 10: ''	List comprehension attempted int('') because ''.split(',') yields [''].
total_credits(text)	text = '4,,3' or '4,3,'	ValueError: invalid literal for int() with base 10: ''	Trailing or consecutive commas produce empty tokens that fail int() conversion.
total_credits(text)	text = 'four,three'	ValueError: invalid literal for int()	Non-numeric characters crash generator expression with unhandled ValueError.
parse_student_record(line)	line = None	AttributeError: 'NoneType' has no attribute 'split'	Invoked .split('|') directly on None without defensive check.
parse_student_record(line)	line = ''	IndexError: list index out of range	Accessed parts[1] and parts[2] when ''.split('|') yields only 1 element.
parse_student_record(line)	line = 'PES1UG24AM428|Asha'	IndexError: list index out of range	Crashed on parts[2] when optional status field was omitted.

4. Defensive Hardening & Code Implementation

To ensure production resilience, course_registration_parser.py was refactored using defensive programming principles. The module now guarantees that no input type, empty string, malformed delimiter, or out-of-range value can trigger an unhandled exception.

Listing: course_registration_parser.py (Defensive Hardened Functions)

```
# --- Hardened Defensive Implementation (course_registration_parser.py) ---
def format_course_code(code):
    if code is None or not isinstance(code, str):
        return ""
    parts = code.strip().split()
    if not parts:
        return ""
    if len(parts) == 1:
```

```
        return parts[0].upper()
    dept = parts[0].upper()
    number = "".join(parts[1:])
    return dept + number

def total_credits(text):
    if text is None or not isinstance(text, str):
        return 0
    total = 0
    for item in text.split(','):
        cleaned = item.strip()
        if not cleaned:
            continue
        try:
            total += int(cleaned)
        except (ValueError, TypeError):
            continue
    return total

def parse_student_record(line):
    default_record = {"srn": "", "name": "", "status": ""}
    if line is None or not isinstance(line, str):
        return default_record
    parts = [p.strip() for p in line.split('|')]
    srn = parts[0].upper() if len(parts) > 0 else ""
    name = parts[1] if len(parts) > 1 else ""
    status = parts[2].lower() if len(parts) > 2 else ""
    return {"srn": srn, "name": name, "status": status}
```

Listing: test_course_registration_parser.py (Fuzzing Suite)

```
# --- Hypothesis Property-Based Fuzzing Suite (test_course_registration_parser.py) ---
from hypothesis import given, strategies as st
from course_registration_parser import format_course_code, total_credits, parse_student_record

@given(st.text() | st.none())
def test_format_course_code_no_crash(s):
    try:
        format_course_code(s)
    except Exception as e:
        assert False, f"format_course_code crashed with: {e}"

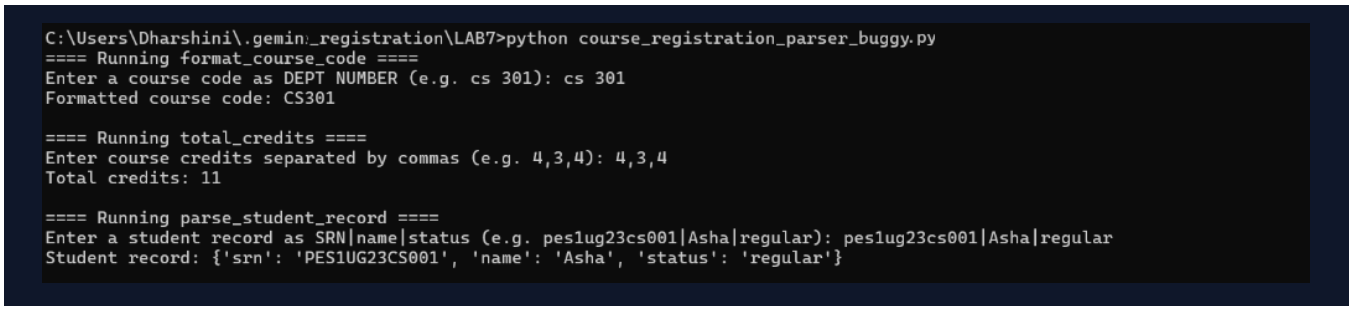
@given(st.text() | st.none())
def test_total_credits_no_crash(s):
    try:
        total_credits(s)
    except Exception as e:
        assert False, f"total_credits crashed with: {e}"

@given(st.text() | st.none())
def test_parse_student_record_no_crash(s):
    try:
        parse_student_record(s)
    except Exception as e:
        assert False, f"parse_student_record crashed with: {e}"
```

5. Verification Results & Terminal Screenshots

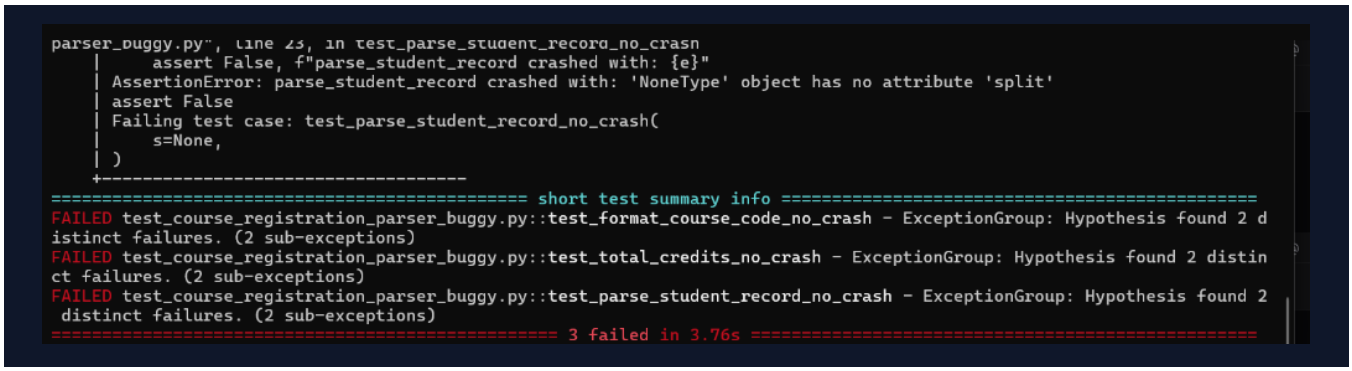
The following screenshots document the execution stages of Lab 7: running the initial buggy script, observing Hypothesis uncover falsifying examples, executing the hardened script, and verifying that all property-based fuzz tests pass with zero crashes.

SS1: Buggy Module Execution with Sample Inputs



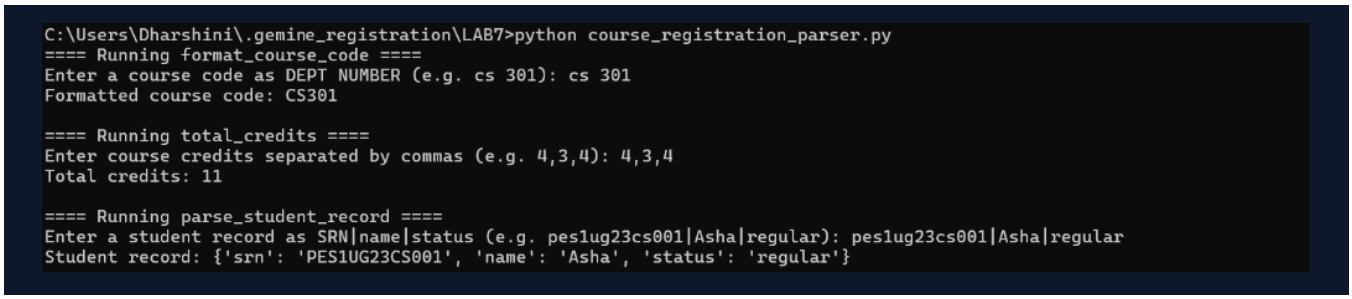
Screenshot 1: python course_registration_parser.py executed with standard inputs.

SS2: Hypothesis Fuzz Testing Failures on Buggy Code



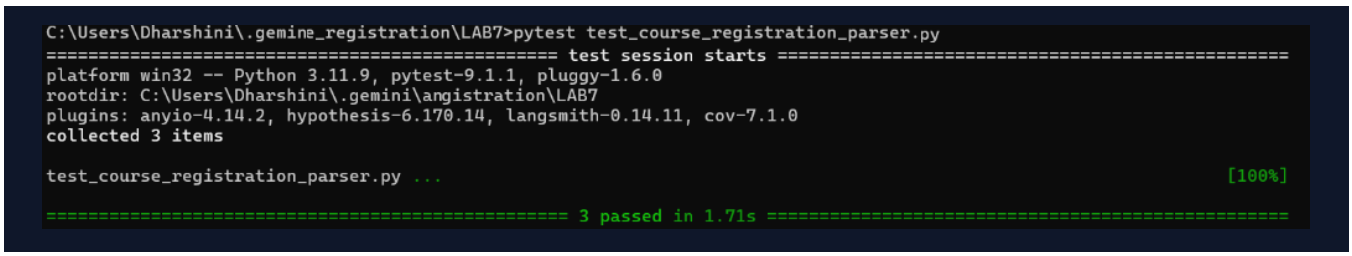
Screenshot 2: pytest test_course_registration_parser.py showing 3 failed tests with Hypothesis shrinking.

SS3: Fixed Module Execution with Sample Inputs



Screenshot 3: python course_registration_parser.py executed on fixed hardened module.

SS4: Hypothesis Fuzz Testing Success on Fixed Code



Screenshot 4: pytest test_course_registration_parser.py showing all 3 tests passed with 0 crashes.

6. Answers to Lab 7 Reflection Questions

Question 1: What types of bugs or vulnerabilities did fuzz testing uncover that standard example-based manual unit tests easily overlook?

Answer:
Standard example-based unit testing suffers from developer confirmation bias: developers author tests based on how they envisage the software being used. As a consequence, manual tests focus on nominal 'happy paths' (e.g. 'cs 301', '4,3,4', and complete student records) or trivial boundaries.

In contrast, property-based fuzz testing via Hypothesis generated hundreds of unexpected edge-case inputs that human testers routinely overlook. Specifically, fuzzing uncovered:
1. Unhandled Type Exceptions: Passing None or non-string values directly crashed .split() with AttributeError.
2. Empty and Malformed String Substrata: Splitting an empty string '' produced a list with one empty element [''], which caused int('') to raise ValueError in total_credits, and unpacking dept, number = code.split() to raise ValueError: not enough values to unpack.
3. Delimiter Inconsistencies: Missing fields in pipe-separated inputs (e.g. 'pes123' or single '|' without remaining components) triggered IndexError: list index out of range on parts[1] and parts[2].
4. Token Malformations: Trailing commas ('4,3,'), internal double commas ('4,,3'), or non-numeric tokens ('four') completely broke list comprehensions.
Fuzz testing systematically exposes how naive assumptions regarding data shape lead to catastrophic service interruptions.

Question 2: How does Hypothesis's 'test case shrinking' mechanism assist developers during debugging and root-cause analysis?

Answer:
When traditional fuzzers locate an input that causes a program crash, the resulting test case is frequently a massive, arbitrary sequence of dozens or hundreds of characters (e.g., 'A8@#\x00\n...\u200b99!'). Diagnosing which exact character or structural property triggered the exception in such noisy data requires laborious manual isolation.

Hypothesis solves this challenge through automated Test Case Shrinking. Once Hypothesis encounters an input that violates an assertion or triggers an unhandled exception, it pauses test generation and initiates a targeted minimization algorithm. It systematically simplifies the failing input—deleting characters, zeroing integers, converting uppercase to lowercase, stripping Unicode to ASCII, and removing substrings—while re-executing the test to ensure the simplified input still triggers the failure.

In our lab run, when format_course_code failed, Hypothesis shrunk the failing input all the way down to s = '' (empty string) and s = None. For parse_student_record, it minimized the crash to s = ''. By presenting the smallest possible falsifying example, Hypothesis immediately pointed to the exact missing guard (e.g. checking if len(parts) > 1), reducing root-cause debugging time from hours to seconds.

Question 3: How does defensive programming and input validation prevent application crashes and protect backend systems from Denial of Service (DoS) or unexpected exploits?

Answer:
In modern distributed architectures and web services, unhandled exceptions in backend parsing logic constitute critical vulnerabilities:
1. Denial of Service (DoS): In web applications, unhandled exceptions like ValueError, IndexError, or AttributeError bubble up to the application framework, resulting in HTTP 500 Internal Server Errors, worker process terminations, and thread crashes. An adversary can continuously send crafted malformed inputs (e.g., empty payloads, lone delimiters, or non-numeric credit values) to crash server processes, exhaust worker pools, and deny availability to legitimate users.
2. Cascading Failures: When parsing routines within data pipelines crash, entire batch ingestion jobs and downstream microservices halt, corrupting transactional state and demanding manual human intervention.
3. Information Disclosure: Unhandled exceptions often dump verbose stack traces to client interfaces, exposing internal file paths, framework versions, and source code structures to potential attackers.

By enforcing defensive programming—employing explicit type checking (isinstance), None-guards, length/bounds verification before index access, and resilient token-level exception handling (try-except blocks that skip malformed tokens)—software components maintain strict crash immunity. Defensive validation ensures systems fail gracefully, return deterministic defaults, and preserve high availability under hostile or corrupted data streams.

7. Version Control Deliverables & Git Audit

Commit Hash	Phase / Stage	Commit Message Summary	Key Deliverables & Changes
Commit 1	Hypothesis Fuzzing Tests	Add Hypothesis property-based fuzz testing suite for course registration parser	Implemented test_course_registration_parser.py with @given(st.text() | st.none()) for all 3 functions.
Commit 2	Defensive Code Hardening	Harden course_registration_parser against malformed inputs and add Lab 7 submission reports	Implemented robust defensive guards in course_registration_parser.py and processor.py; generated PDF/DOCX reports.